Django Templates and Static Files

Django Templates:

A template in Django is basically written in HTML, CSS, and JavaScript in a .html file.

- It is not recommended to write html code inside python script (views.py file) because:
- It reduces readability because Python code mixed with html code
- No separation of roles. Python developer has to concentrate on both python code and HTML Code.
- It does not promote reusability of code. We can overcome these problems by separating html code into a separate html file.

From the Python file (views.py file) we can use these templates based on our requirement. We have to write templates at project level only once and we can use these in multiple applications.

Steps to develop Template based Application:

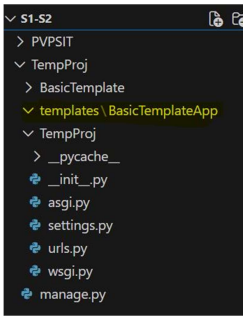
1. Creation of Project

   i. django-admin startproject name_of_the_project or
   ii. python –m django startproject name_of_the_project
   Ex: python -m django startproject TempProj

2. Creation of Application

   i. python manage.py startapp BasicTemplate
   Ex:
   (PVPSIT) PS F:\S1-S2\TempProj> python manage.py startapp BasicTemplate

3. Add this application to the project in settings.py file, so that Django aware of the application




```
INSTALLED_APPS = [
'django.contrib.admin',
'django.contrib.auth',
'django.contrib.contenttypes',
'django.contrib.sessions',
'django.contrib.messages',
'django.contrib.staticfiles',
'BasicTemplate',
]
```

4. Create a 'templates' folder inside main project folder. In that templates folder create a separate folder named with 'BasicTemplateApp' to hold that particular application specific templates. Ex: F:\S1-S2\TempProj\templates\BasicTemplateApp

Suresh Yadlapati

5. Add templates folder to settings.py file so that Django can aware of our templates. and also check 'APP_DIRS': True,

```
TEMPLATES = [
    {
        ….
        'DIRS': ['F:\S1-S2\TempProj\\templates'],
        ….
    },
]
```

6. Create html file inside BasicTemplateApp folder. This html file is nothing but template. Ex. Sample.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>First Template Page</title>
  </head>
  <body>
    <h1>Hello welcome to Second Hero of MVT: Templates</h1>
  </body>
</html>
```

7. Define Function based view inside views.py file

```
django.shortcuts import render
# Create your views here.
def templateview(request):
        return render(request,'BasicTemplateApp/Sample.html')
                #render() function sends data to HTML template.
```

8. Define URL Pattern either at project or application level(we have to create urls.py) by importing views of application BasicTemplate.

```
from BasicTemplate import views
urlpatterns = [
  path('admin/', admin.site.urls),
   path('FirstTemplate/', views.templateview),
]
```

9. Run Server and send Request: python .\manage.py runserver

SureshYadlapati

Template Variables and Template Tags:

In Django, dynamic content is passed from **views.py** to the HTML template.

Django templates allow us to:

- Display dynamic data
- Perform basic programming logic
- Control how data appears in HTML

Template Variable: Template variables are used to display data in HTML.

Syntax: {{ variable_name}}

Django application to send Date and Time from views.py to Template File:

DateTime.html - in templates folder

```
<!DOCTYPE html>
<html>
  <head>
    <title>First Template Page</title>
  </head>
  <body>
    <h1> Current Date and Time : {{insert_date}} </h1>
  </body>
</html>
```

views.py - in app views.py file

```
import datetime
def DTView(request):
    date=datetime.datetime.now()
    my_dict={'insert_date':date}
    return render(request,'BasicTemplateApp/DateTime.html',context=my_dict)
    #context=my_dict → passes dictionary data to template.
```

urls.py - in project urls.py file

```
from django.contrib import admin
from django.urls import path
from BasicTemplate import views
urlpatterns = [
    path('admin/', admin.site.urls),
    path('TemplateSample/', views.FirstView),
    path('DT/', views.DTView),

]
```

Suresh Yadlapati

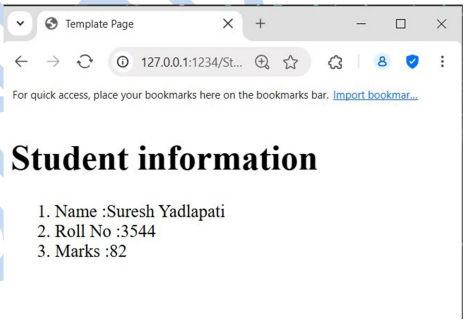
Django Application to display student information:

views. py

```
def StudentView(request):
    name='Suresh Yadlapati'
    rollno=3544
    marks=82
    my_dict1={'name':name,'rollno':rollno,'marks':marks}
    return render(request,'BasicTemplateApp/StudentInfo.html',context=my_dict1)
```

StudentInfo.html

```
<html>
    <head>
        <title>Template Page</title>
    </head>
    <body>
        <h1> Student information</h1>
        <ol>
            <li> Name :{{name}}</li>
            <li> Roll No :{{rollno}}</li>
            <li> Marks :{{marks}}</li>
        </ol>
    </body>
</html>
```



urls. py

```
path('StudentInfo/', views.StudentView),
```

Template Tag:

- A **Template Tag** is a special syntax used inside HTML to perform logic in templates.
- In tags , we can perform programming logic like executing if statements and for loops.
- These keywords, if and for, are called "template tags" in Django.
- To execute template tags, we surround them in {% %} brackets.

If template tag sytax:

```
{% if condition %}
        statements
{% else %}
        statements
{% endif %}
```

```
{% if greeting == 1 %}
<h1>Hello</h1>
{% else %}
<h1>Bye</h1>
{% endif %}
```

Suresh Yadlapati

For template tag syntax:

```
{% for item in list %}
        statements
{% endfor %}
```

```
{% for i in numbers %}
   <p>{{ i }}</p>
{% endfor %}
```

Develop a Django Application to Display Student Information Using Template Tags

views.py

```
def StudentResultView(request):
   name = 'Suresh Yadlapati'
   rollno = 3544
   marks = 82
   subjects = ['Python', 'Django', 'DBMS']
   my_dict2 = {
      'name': name,
      'rollno': rollno,
      'marks': marks,
      'subjects': subjects
   }
   return render(request, 'BasicTemplateApp/StudentResults.html', context=my_dict2)
```

StudentResults.html (Inside of F:\S1-S2\TempProj\templates\BasicTemplateApp)

```
<html>
        <head>
                <title>Student Info</title>
        </head>
<body>
         <h1>Student Details</h1>
        <p>Name: {{ name }}</p>
         <p>Roll No: {{ rollno }}</p>
        <p>Marks: {{ marks }}</p>
        <!-- Using IF Template Tag -->
        {% if marks >= 35 %}
                <h3 style="color:green;">Result: Pass</h3>
        {% else %}
                <h3 style="color:red;">Result: Fail</h3>
        {% endif %}
        <h2>Subjects List</h2>
         <!-- Using FOR Template Tag -->
        {% for sub in subjects %}
                 <p>{{ sub }}</p>
        {% endfor %}
        </body>
</html>
```

Student Details

Name: Suresh Yadlapati

Roll No: 3544

Marks: 82

Result: Pass

Subjects List

Python

Django

JAVA

urls.py : path('studentresult/', views.StudentResultView),

Suresh Yadlapati